Vault Markets

Litepaper — Protocol Specification V1.6

Conditional tokens + Hybrid CLOB + CLMM/FPMM liquidity + Risk Engine + Recourse Advances

Date: February 5, 2026
Status: Draft for engineering handoff

Disclaimer. This document describes a software protocol design and is not legal, tax, or investment advice. Real-money prediction markets may trigger regulatory obligations depending on jurisdiction and implementation details.

Abstract

Vault Markets is a creator- and curator-native prediction market protocol designed to bootstrap deep early liquidity without unbacked "virtual" reserves. V1.6 combines (i) fully-collateralized multi-outcome conditional shares, (ii) a hybrid central-limit order book (CLOB) for price discovery, (iii) a capital-efficient concentrated-liquidity layer (CLMM) plus an always-on backstop AMM (FPMM), (iv) an explicit Market Maker Risk Engine to defend protocol inventory against loss-versus-rebalancing (LVR) and toxic flow, and (v) recourse advances that let creators and trusted KOLs operate with zero upfront cash while remaining economically accountable through debt-first waterfalls.

Contents

1. Design goals and non-goals

Goals.

- Deep early liquidity without insolvency: all protocol "subsidy" is real collateral deployed as liquidity, not virtual reserves.
- Order book parity: a CLOB is the primary price discovery venue; AMMs are backstops and retail convenience routes.
- Capital efficiency: concentrated liquidity (CLMM) is available where it improves depth per dollar.
- Treasury defense: dynamic surge fees, cooldown/hysteresis, and inventory skew apply when flow consumes protocol inventory.
- Creator/KOL distribution with accountability: credit lines + recourse debt + strict debt-first waterfalls.

Non-goals (V1.6).

- Permissionless resolution (V1.6 uses admin/multisig with evidence and dispute window).
- Full on-chain matching (V1.6 uses off-chain matching with on-chain settlement for the CLOB).
- Governance token mechanics (tokenization can be layered later; V1.6 focuses on unit economics and solvency).

2. System overview

Actors.

- **Traders:** place CLOB orders or execute swaps via the router.
- **Creators:** propose markets and receive creator fee streams (escrowed until finality).
- **KOLs/Curators (trusted):** provide range guidance and may operate liquidity programs under credit lines.
- **Protocol/Treasury:** provides senior seed liquidity, runs risk policies, and collects protocol fees.
- **Protocol Market Maker (PMM, optional):** posts quotes on the CLOB under Risk Engine control.

Core assets.

- **USDC:** the sole collateral asset for real-money markets.
- **Outcome shares T_i (ERC-1155):** fully-collateralized conditional shares redeemable for 1 USDC if outcome i wins, else 0.
- **Complete set:** 1 unit of each outcome share; can be merged back into 1 USDC pre-resolution.
- **LP/vLP shares:** represent ownership of protocol-deployed liquidity (FPMM LP or CLMM position-vault shares).
- **ProfileID:** identity-bound account used for credit lines and recourse accounting (creators and trusted KOLs).

High-level architecture.

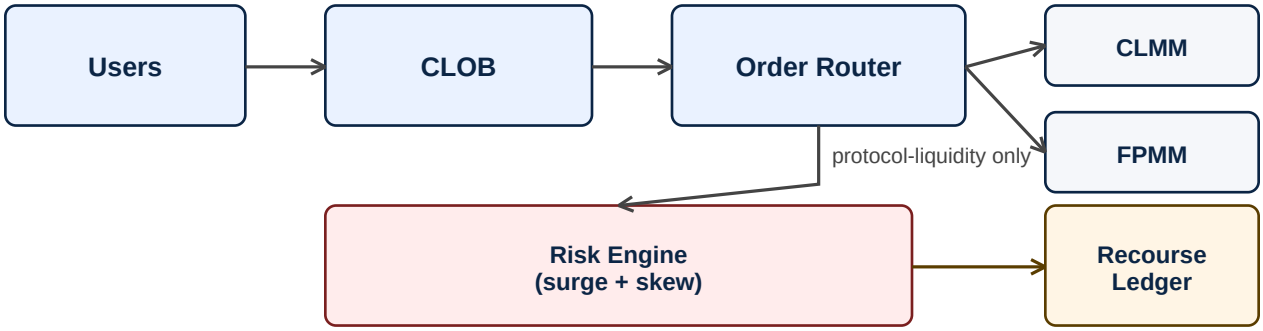

Figure 1. Users trade primarily on the CLOB; the router accesses CLMM and FPMM backstops. The Risk Engine applies only to protocol-exposed liquidity. Recourse ledgers track credit lines and debt-first repayment.

3. Conditional token model and solvency

Each market has N outcomes. For outcome i, an ERC-1155 share token T_i represents a claim that redeems for 1.00 USDC if outcome i is the winner. Shares are minted only by splitting USDC into complete sets and can be merged back to USDC by burning a complete set. This ensures solvency by construction: the system never creates uncollateralized payout obligations.

Split/Merge primitives.

```
Split(x USDC)  -> mint x shares of each outcome token T_1..T_N
Merge(x of each T_i) -> redeem x USDC
```

Resolution and redemption.

V1.6 resolution is performed by an admin/multisig with an evidence URI and a fixed dispute window. After finality, winning shares redeem 1 USDC; losing shares redeem 0. Share supplies remain fully collateralized throughout.

4. Hybrid market microstructure

Primary venue: CLOB.

The CLOB is the primary price discovery venue. Orders are signed (EIP-712) and matched off-chain; settlements occur on-chain in batches. User-to-user matches do not expose protocol inventory unless a protocol market maker is the counterparty.

Backstops: CLMM and FPMM.

- **CLMM (per outcome):** concentrated liquidity pool trading T_i vs USDC for capital-efficient depth near the current price. CLMM can go out-of-range.

- **FPMM (multi-outcome):** always-on constant-product pool over outcome shares, ensuring continuous liquidity even when CLMM is out-of-range.
- **Order Router:** routes fills CLOB-first, then CLMM, then FPMM under user slippage and protocol impact constraints.

Protocol exposure boundary.

Risk Engine controls (surge fees, skew, spread/size throttles) apply only when a trade consumes protocol-provided liquidity (protocol CLMM, protocol FPMM, or protocol PMM quotes). This defends treasury without breaking peer price discovery.

5. Liquidity bootstrapping and hardening (seed as a loan)

Markets may receive senior seed liquidity P0 funded by the protocol treasury and/or by a creator treasury pool. Seed liquidity is deployed into a CLMM ladder (wide/core/tight bands) plus an optional FPMM baseline. A portion of trading fees is routed to repay outstanding principal P; as repayment occurs, ownership of the seed liquidity (LP/vLP shares) transfers from the lender to the protocol/creator ("hardening").

Hardening invariant: liquidity depth does not cliff.

Hardening changes ownership of liquidity rather than removing liquidity abruptly. This avoids sudden depth collapses at graduation.

```
At each epoch:
  r = min(P, hardening_fees_available)
  P <- P - r
  Transfer fraction r/P0 of seed LP (or vLP) from lender to owner
```

6. Credit lines and recourse advances (creators + trusted KOLs)

V1.6 supports identity-bound credit lines for creators and trusted KOLs/curators. Credit is issued to a ProfileID (not to a raw wallet) to reduce sybil-creator risk. Profiles can launch markets or run liquidity programs with zero upfront cash, but their future earnings are junior: all fee streams owed to the profile are seized until recourse debt is repaid.

AdvanceAccount state per ProfileID u.

```
L_u : credit limit (USDC)
D_u : outstanding recourse debt (USDC)
tier/status flags (Trusted KOL, Verified Creator, Public)
authorized signer keys / wallets
```

Debt-first waterfall.

```
On fee release (after market finality):
  r = min(D_u, F_u)
  D_u <- D_u - r
  payout_u = F_u - r
  (100% seizure until D_u = 0)
```

How recourse debt is created.

- **Junior loss reserve consumption:** protocol provides a junior first-loss buffer J for a market or liquidity program; realized MM losses consume J first; consumed amount becomes recourse debt assigned to one or more profiles via a configurable allocation vector.
- **Program budget advances:** trusted KOLs can request additional tight-band CLMM or PMM quote budgets under their credit lines; if these programs lose money, losses accrue as recourse debt.
- **Debt-first applies to both creators and trusted KOLs:** increasing a KOL credit line increases capability, but the debt-first rule preserves discipline.

Credit line increases.

Credit limits L_u can be increased for trusted KOLs based on performance metrics (repayment history, dispute rates, realized MM loss rates, and operational compliance). Increases are gated by ProfileID verification and can be reduced automatically when risk thresholds are breached.

7. Risk Engine (dynamic surge fees, cooldown, inventory skew)

The Risk Engine defends protocol inventory against LVR and toxic flow by dynamically adjusting the effective fee and (optionally) PMM quoting parameters. All computations in the hot path are O(1) and designed for high-volume L2 execution.

7.1 Block-decay velocity metric (O(1)).

```
State:
  V          decayed protocol-exposed notional (USDC)
  lastBlock  last update block
Update on protocol-liquidity execution:
  Δ = block - lastBlock
  V_next = V * powAlpha[Δ] + X_t
  V <- V_next
  lastBlock <- block
```

7.2 Surge multiplier with cooldown (anti-oscillation).

```
Compute ratio using post-trade state (anti-splitting):
  r_next = V_next / V_ref
Lookup target multiplier (interpolated LUT):
  M_target = interpSurgeLUT(r_next)
Cooldown/hysteresis update:
  M <- max(M_target, M * powBeta[Δ])   // decreases slowly
  M <- max(M, M_target)               // increases immediately
```

7.3 Inventory skew.

Let q_i be protocol-held inventory of shares for outcome i across CLMM/FPMM/PMM. Let r_i be the reference mid price (CLOB mid preferred). Define value weights I_i = q_i * r_i and inventory share s_i = I_i / sum(I). Imbalance z_i = s_i - 1/N. When a trade would increase exposure to an already-heavy outcome, the engine applies a protective skew (worse pricing / higher effective fee).

```
Directional skew multiplier:
```

```
  dir = +1 (user buys outcome j from protocol)
  dir = -1 (user sells outcome j to protocol)
  m_inv = expApprox(-gamma * dir * z_j)   // implemented via LUT/piecewise
```

7.4 Effective fee.

```
Baseline trading fee: f0 = 3.00% (300 bps)
Protocol-liquidity effective fee:
  f_eff = clip( f0 * M * m_inv, f_min, f_max )
Note: f_eff applies only when consuming protocol liquidity.
```

8. Trusted range council and CLMM guardrails

V1.6 uses a trusted KOL/curator range council and internal research to set initial CLMM ranges. We intentionally avoid complex on-chain voting and commit-reveal mechanics in V1.6. Instead, the protocol constrains range risk via mechanical guardrails that limit the blast radius of mis-ranging.

Guardrails (enforced on-chain).

- Always-on wide band: a minimum liquidity allocation to a wide safety range that remains active under large moves.
- Minimum band width: tight bands cannot be narrower than w_min.
- Allocation caps: tight-band allocation cannot exceed alpha_max of protocol liquidity for the market.
- Rate-limited re-centering: ranges cannot be changed faster than a configured cadence; emergency pull is allowed under surge regime.
- Surge-aware de-risking: when the Risk Engine enters a high-M regime, the router reduces reliance on tight CLMM and widens bands.

Optional accountability coupling.

If a trusted KOL's credit line funds a liquidity program (tight-band CLMM or PMM quote budget), the program's realized losses can be attributed to that ProfileID as recourse debt. This provides economic discipline without requiring staked voting.

9. Fees, routing, and debt-first waterfalls (3% baseline)

V1.6 uses a baseline trading fee of 3.00% (300 bps). The system supports venue-specific splits (maker rebate programs, protocol surcharges) while maintaining a consistent user-level fee policy.

Fee surfaces.

- CLOB fills: 3% trading fee applied at settlement; maker rebates may be funded from protocol budget to maintain tight spreads.
- CLMM swaps: pool fee tier may be set low, with router collecting the 3% fee to avoid double-fees; or pool fee can be 3% with zero router surcharge.
- FPMM swaps: baseline 3% fee; protocol-liquidity routes may apply f_eff > 3% under surge/skew defenses.

Waterfalls.

```
1) HardeningEscrow: repay senior seed principal P (while P > 0)
2) Protocol revenue
3) Creator/KOL fee escrows accrue (released after resolution finality)
4) On release: repay ProfileID debt D_u first (100% seizure) then pay remainder
```

10. Gas and throughput engineering requirements

Even on an L2, high-volume markets can incur meaningful costs. V1.6 therefore makes gas strategy a first-class requirement. The objective is an O(1) hot path per protocol-liquidity execution and batch-oriented settlement for CLOB volume.

10.1 LUT mandate and interpolation defense.

The Risk Engine MUST NOT evaluate exp(), pow(), or log() in the hot path. It MUST use lookup tables (LUTs) with piecewise linear interpolation between LUT nodes to avoid step-function discontinuities that can be exploited by order splitting or boundary MEV.

```
Interpolated LUT evaluation:
  x = (r - 1.0)
  i = floor(x / Δr)
  t = (x - i*Δr) / Δr   // in [0,1]
  M(r) = (1 - t)*LUT[i] + t*LUT[i+1]
Compute with r_next (post-trade velocity) to internalize impact.
```

10.2 Storage-write minimization.

- Per protocol-liquidity fill, update at most: V, lastBlock, M, and z_j for the traded outcome.
- Avoid loops over all outcomes in the hot path. Inventory accounting should be incremental and outcome-local.
- Use packed structs and fixed-point math (mulDiv) to reduce storage and prevent overflow.

10.3 Batch settlement and netting.

- Settle many CLOB matches per transaction (batch).
- Net USDC transfers per participant where possible to reduce ERC20 transfers.
- Use ERC-1155 batchTransfer for multi-fill outcome share movements.

11. Security considerations and failure modes

- **LVR / news shocks:** defended via surge multipliers with cooldown and protocol-liquidity-only application.
- **Toxic flow:** defended via inventory skew (charge risk premia when adding to an imbalanced book).
- **Mis-ranging CLMM:** contained via wide-band minimum allocation, tight-band caps, and surge-aware de-risking.
- **Sybil creators / abandoned debts:** mitigated via ProfileID identity binding and credit-limit gating.

- **Oracle / resolution risk:** mitigated via evidence requirements, dispute window, and creator/KOL fee escrows until finality.
- **MEV / boundary exploits:** mitigated via interpolated LUTs and use of post-trade velocity state (V_next).

V1.6 is designed to remain solvent under adversarial flow, but it is not a guarantee of profitability. Protocol liquidity programs should be launched with conservative caps and iterated using real-world telemetry.

12. Recommended parameters (initial defaults)

Parameter	Default	Notes
f0 (baseline fee)	300 bps	3% trading fee baseline
f_max (protocol routes)	1500 bps	cap under surge/skew (example: 15%)
alpha (velocity decay)	0.97 per block	L2-dependent; use LUT powAlpha[Δ]
beta (cooldown decay)	0.995 per block	slow decay to avoid oscillation
Δr (surge LUT step)	0.05	use interpolation between points
r_max (surge LUT cap)	10.0	cap multiplier at r_max
gamma (inventory skew)	tuned	controls aggressiveness of skew
alpha_wide (CLMM wide band)	>= 20%	minimum safety liquidity allocation
alpha_max (tight bands)	<= 40%	cap tight-band allocation
w_min (min band width)	market class	wider for volatile/live events
Credit line L (creator)	$500-$2k	tiered by verification + performance
Credit line L (trusted KOL)	$1k-$10k	tiered; debt-first repayment

Appendix A. FPMM math (multi-outcome constant product)

The backstop AMM is a multi-outcome constant-product pool over outcome share balances b_i. Shares are minted via Split(x) and merged via Merge(x). Prices are derived from the reciprocal balance weights.

```
State: b_i = pool balance of outcome share T_i
Invariant: K = Π_i b_i
Implied price/probability (marginal): p_i = (1/b_i) / Σ_k (1/b_k)
```

Buy outcome j with x USDC (after fee).

```
x_net = x * (1 - fee)
Mint complete sets from x_net, add x_net to each b_i
Remove Δ_j shares of outcome j such that:
  (b_j + x_net - Δ_j) * Π_{i≠j}(b_i + x_net) = Π_i b_i
Closed form:
  Δ_j = b_j + x_net - K / Π_{i≠j}(b_i + x_net)
```

Sell s shares of outcome j for USDC.

```
Find y (gross USDC) such that:
  (b_j + s - y) * Π_{i≠j}(b_i - y) = K
Constraints: 0 ≤ y < min_{i≠j} b_i
Solve y via bounded binary search. User receives y*(1 - fee).
```

Appendix B. CLMM math (concentrated liquidity)

For each outcome i, the CLMM pool trades T_i against USDC. We use standard square-root price notation. CLMM positions are held in a vault and fractionalized by vLP shares for ownership transfer and accounting.

```
Let P = USDC per share, s = sqrt(P)
Range bounds: s_a = sqrt(P_a), s_b = sqrt(P_b)
Liquidity: L
If P_a < P < P_b:
  shares Δx = L * (s_b - s) / (s * s_b)
  USDC  Δy = L * (s - s_a)
```

vLP ownership transfer.

CLMM position NFTs are held by CLMMPositionVault. Fungible vLP shares represent fractional ownership. Hardening transfers vLP pro-rata with principal repayment, keeping liquidity depth stable while changing ownership.

Appendix C. LUT and interpolation specification (anti-step MEV)

LUTs are used for decay powers and surge multipliers. To avoid exploitable discontinuities, surge multipliers must be evaluated with linear interpolation between LUT nodes.

```
Given r in [1.0, r_max], step Δr:
  i = floor((r - 1.0)/Δr)
  t = ((r - 1.0) - i*Δr)/Δr
  M = (1 - t)*LUT[i] + t*LUT[i+1]
Use r_next = V_next / V_ref (post-trade) to reduce order-splitting exploits.
```

Reference pseudocode (fixed-point).

```
interpLUT(rWad):
  if rWad <= 1e18: return 1e18
  clamp rWad to rMax
  x = rWad - 1e18
  i = x / DR
  t = (x - i*DR) * 1e18 / DR
  return LUT[i] + (LUT[i+1] - LUT[i]) * t / 1e18
```

Appendix D. Accounting definitions (PnL, loss, recourse)

Recourse debt requires a deterministic definition of realized loss attributable to protocol liquidity programs. V1.6 defines realized PnL as the USDC value of protocol-held assets after unwind, including accrued fees, minus deployed principal. Losses consume junior reserves first; consumed amount becomes recourse debt as configured by the market's recourse allocation vector.

```
Let NAV_final be protocol liquidity NAV after unwind/settlement.
Let Principal = deployed senior + junior program capital.
Loss = max(0, Principal - NAV_final)
JuniorConsumed = min(Loss, JuniorReserve)
For each responsible ProfileID u with allocation a_u:
  D_u <- D_u + a_u * JuniorConsumed
```

NAV_final computation should be implemented with conservative, auditable rules and emitted as events for monitoring and dispute analysis.